

Linked Lists

What is a Linked List

A linked list is a linear data structure where elements (called nodes) are not stored in contiguous memory. Each node holds its data plus a pointer/reference to the next node. The list is accessed starting from a 'head' pointer, and the last node points to None/null, marking the end.

Types

- Singly Linked List: each node points only to the next node (forward traversal only).
- Doubly Linked List: each node points to both the next and previous node (traversal in both directions).
- Circular Linked List: the last node points back to the head instead of None, forming a loop.

Time Complexity

- Access by index: $O(n)$ — must traverse from the head, no direct indexing.
- Search: $O(n)$.
- Insertion at head: $O(1)$ — just repoint the head.
- Insertion at tail: $O(n)$ for singly linked list without a tail pointer, $O(1)$ if a tail pointer is maintained.
- Deletion: $O(1)$ if you already have a reference to the node before it, otherwise $O(n)$ to find it first.

Linked List vs Array

- Arrays offer $O(1)$ random access; linked lists require $O(n)$ traversal to reach a specific position.
- Linked lists offer $O(1)$ insertion/deletion once you're at the right node; arrays need $O(n)$ shifting.
- Arrays have fixed size in low-level languages; linked lists grow and shrink dynamically with no resizing cost.
- Linked lists use extra memory per element to store the pointer(s).

Implementation in Python

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
```

```
def insert_at_head(self, data):
```

```
    new_node = Node(data)
```

```
    new_node.next = self.head
```

```
    self.head = new_node
```

```
def insert_at_tail(self, data):
```

```
    new_node = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new_node
```

```
    return
```

```
    current = self.head
```

```
    while current.next:
```

```
        current = current.next
```

```
    current.next = new_node
```

```
def print_list(self):
```

```
    current = self.head
```

```
    while current:
```

```
        print(current.data, end=" -> ")
```

```
        current = current.next
```

```
    print("None")
```

Common Interview Problems

- Reverse a linked list (iterative and recursive).
- Detect a cycle using Floyd's Cycle Detection (slow/fast pointers).
- Find the middle node in one pass using slow/fast pointers.
- Merge two sorted linked lists.
- Remove the nth node from the end in one pass.
- Detect and remove a loop, find the starting node of the loop.